Implementing File Systems
— Chapter 11

叶冠华

g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)

北京郵電大學

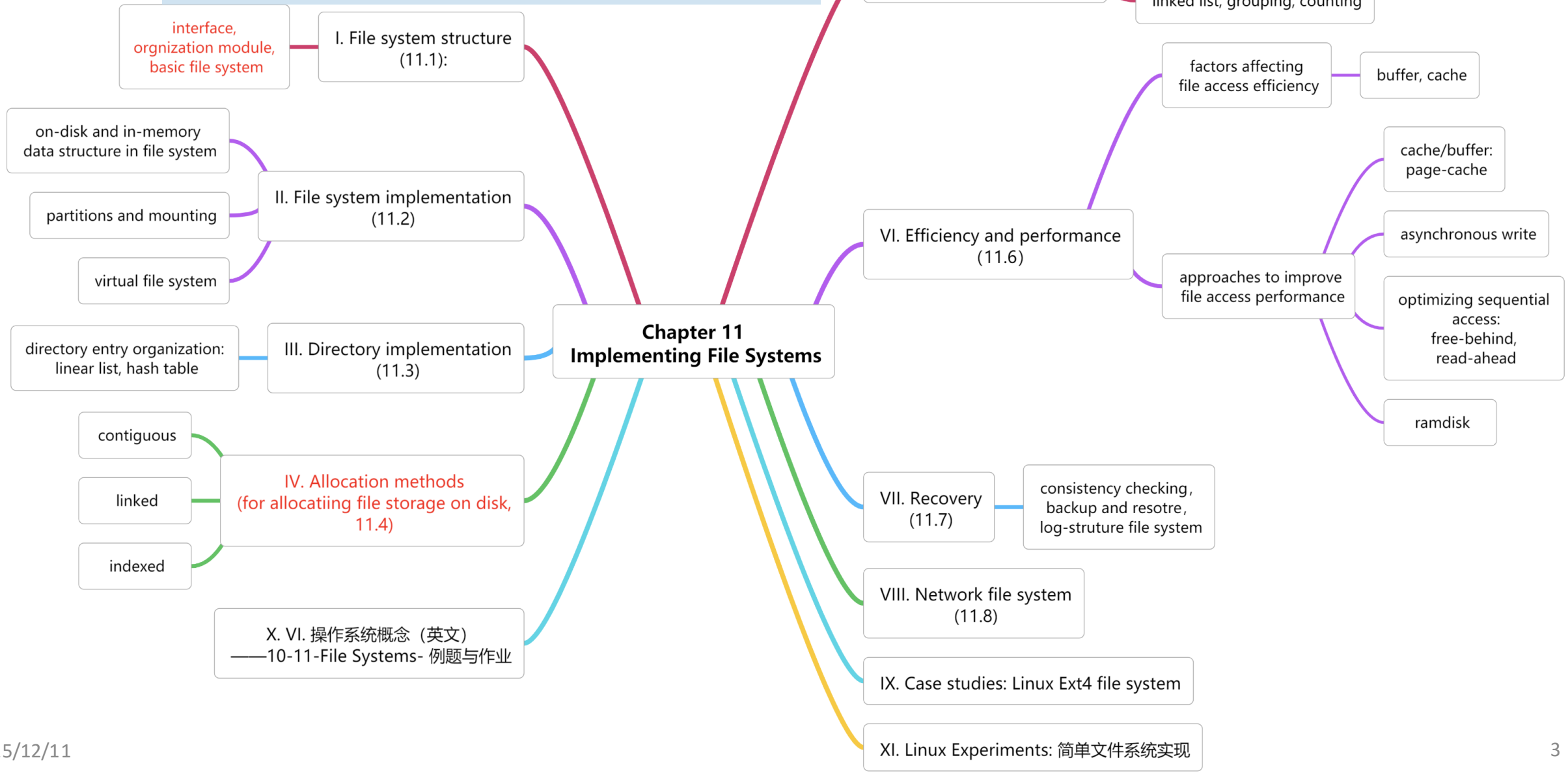
Review

Logical file system

- File
  - File Attribute(=metadata): <Name, Identifier, Type, Location, Protection, …>
  - FCB (File Control Block): A data structure store most attributes of a file
  - File Operation: Create, Read, Write, Delete, Truncate
  - File Lock: Shared lock, Exclusive lock. (Mandatory/advisory)
  - File Access Method: Sequential, Direct, Index-based
- Directory
  - Directory is a table. One row = one directory entry. One column = one file attribute.
  - In DOS, Directory entry=<Name, Identifier, Type, Location, Protection, …>
  - In Linux, Directory entry=<Name, Identifier>, Inode=< Identifier, Type, Location, …>
  - Single-level, Two-level, Tree-structure, Acyclic-graph

CHAPTER OBJECTIVES

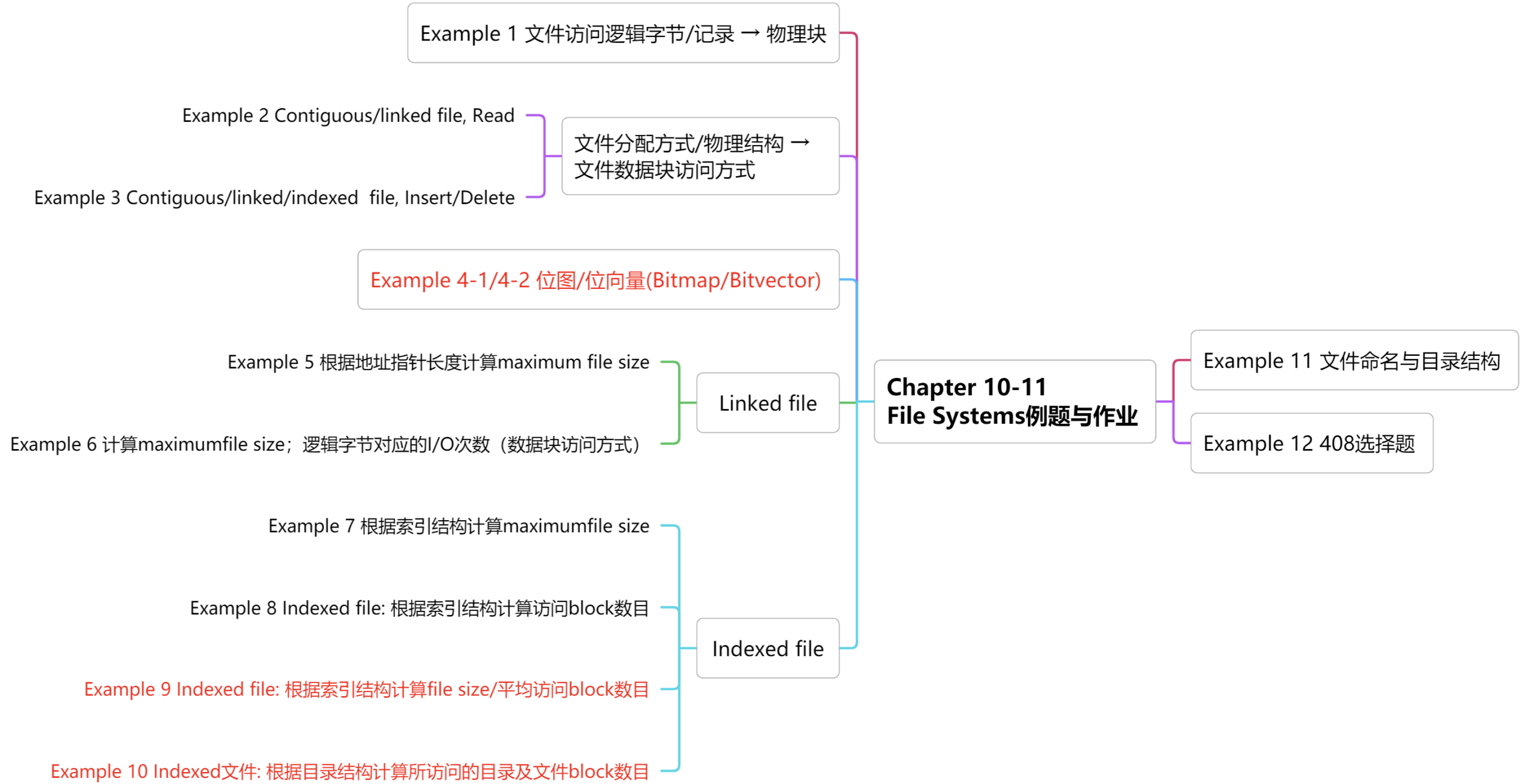
- To describe the details of implementing local file systems and directory structures.
- To describe the implementation of remote file systems.
- To discuss block allocation and free-block algorithms and trade-offs.



Terminology-11

sector, block, Cluster, extent	扇区, 磁盘块, 簇, 区段,扩展块
boot control block, boot block in UFS, partition boot sector in NTFS	引导控制块, UFS引导块, NTFS分区引导扇区
volume/partition control block superblock in UFS master file table in NTFS, mount table	卷/分区控制块, UFS超级块, NTFS主控文件表, 安装表
system-wide open-file table, per-process open-file table file descriptor, file handle	系统级打开文件表, 进程级打开文件表, 文件描述符, 文件句柄
inode, soft link, hard link	索引节点, 软链接, 硬链接

raw partition, root partition	原始/生/空白分区, 根分区
boot loader	引导加载程序
VFS	虚拟文件系统
dentry object	目录条目对象
contiguous allocation, linked allocation, Indexed allocation, File-Allocation Table	连续分配, 链接分配, 索引分配, 文件分配表
bit map/bit vector	位图/位向量
buffer cache, page cache, unified buffer cache	缓冲区缓存, 页面缓存, 统一缓冲区缓存
asynchronous write, free-behind, read-ahead, RAM/virtual disk	异步写, 随后释放, 预先读取, 内存/虚拟硬盘


Example 1 文件访问逻辑字节/记录 → 物理块
Example 2 Contiguous/linked file, Read
文件分配方式/物理结构 → 文件数据块访问方式
Example 3 Contiguous/linked/indexed file, Insert/Delete
Example 4-1/4-2 位图/位向量(Bitmap/Bitvector)
Example 5 根据地址指针长度计算maximum file size
Linked file
Example 6 计算maximumfile size；逻辑字节对应的I/O次数（数据块访问方式）
Example 7 根据索引结构计算maximumfile size
Example 8 Indexed file: 根据索引结构计算访问block数目
Indexed file
Example 9 Indexed file: 根据索引结构计算file size/平均访问block数目
Example 10 Indexed文件: 根据目录结构计算所访问的目录及文件block数目
Chapter 10-11
File Systems例题与作业
Example 11 文件命名与目录结构
Example 12 408选择题

11.1 File System Structure

- I/O transfer between memory and disk is performed in unit of ***block***, which contains one ore more ***sectors***
  - supporting direct/random access and sequential access, through moving read-write head and rotating disk-plate, feasible for accessing any block of information on disk

- Layered file system illustrated in Fig. 11.1

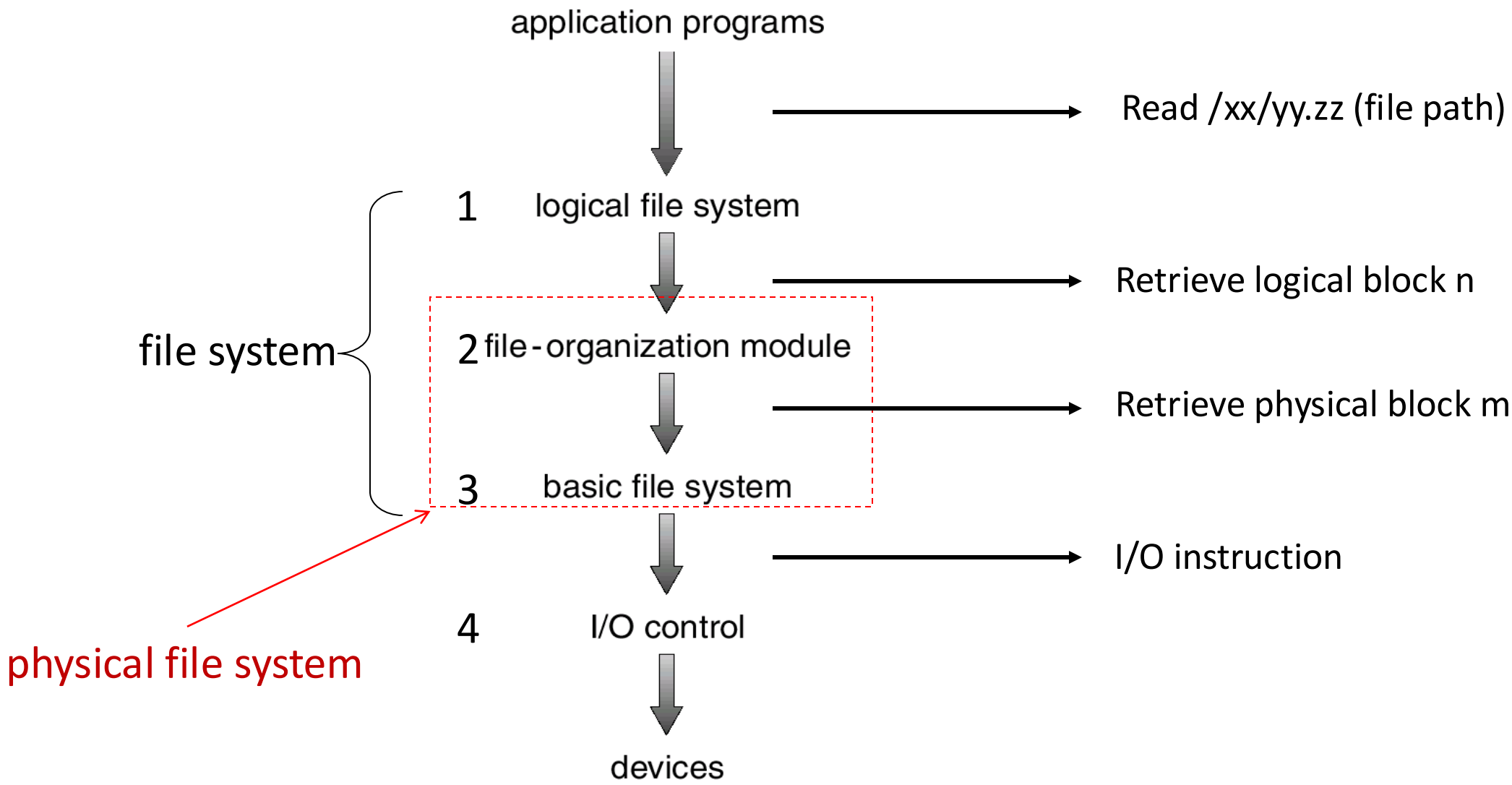
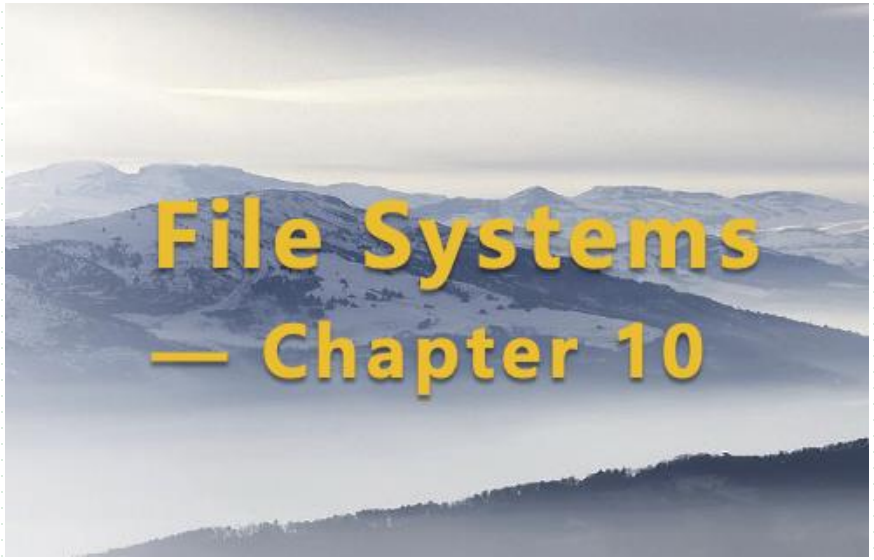


Fig.11.1 Layered File System

Layered File System

- 1 Logical file system (= Chapter 10 File System)
- File
  - File path -> Logical block n
  - …
- Directory
  - …



Layered File System

- 2 File organization module
  - mapping between **logical block/record number** and **physical block address**
  - disk *free space* management
  - disk space allocation for file creation, modification, deletion…

- 3 Basic file system
  - generating generic/high-level I/O commands to device drivers to **read** or **write** physical blocks
  - on the basis of the **block number** or **physical block address**
    - e.g. driver#, cylinder#, track#, sector#

Layered File System

- 4 I/O control, i.e., I/O subsystem in OS kernel, including device drivers and interrupt control
  - conducting information transferring between memory and disk
  - device driver
    - input: high-level I/O commands
    - output: low level hardware-specific I/O instructions

11.2 File System Data Structures

- On-disk structure
  - Boot control block
  - Volume control block
  - Directory structure
  - File control block (FCB)
- In-memory structures
  - Mount table
  - Directory cache
  - System-wide open-file table。
  - Per-process open-file table。
  - Disk buffer

On-disk structure

- Boot control block
  - contain information needed by the system to boot OS from the volume, also known as
    - boot block in **UFS**
    - partition boot sector in **NTFS,**
- Volume control block
  - contain volume/partition details
    - the number of blocks in the partition
    - size of the block
    - free block count, free block pointer
    - free FCB count, free FCB pointer
  - also known as,
    - in UFS, called superblock/超级块
    - in NTFS, stored in the master file table/主控文件表

On-disk Structure

- Directory structure: set of directory entries
  - used to organize file, also known as,
    - in UFS, it includes file names and associated **inode** number, i.e. directory entry=<filename, inode_number>
    - in NTFS, stored in the master file table
- Per-file file control block(FCB)
  - contain many details about the file, including ownership, permission, location, etc.
  - also known as **inode** (i.e. index node) in Unix/Linux file systems
  - has an unique identifier , i.e. inode_number, to allow association with a directory entry
  - in NTFS, this details are stored in the master file table

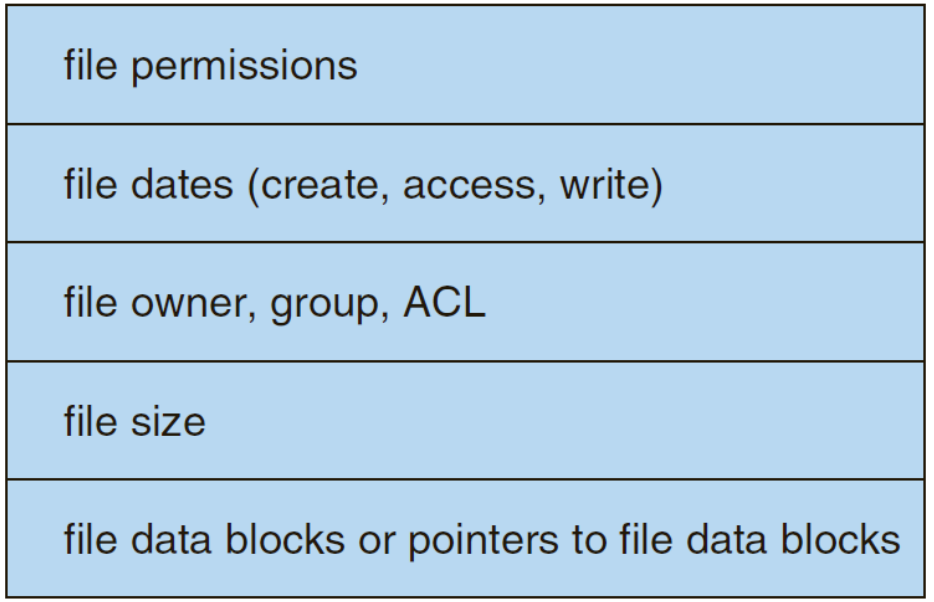


Fig.11.2 A typical file control block FCB, e.g. inode

inode in Linux

- inode包含文件的元信息
  - 文件的字节数
  - 文件拥有者的User ID，文件的Group ID
  - 文件的读、写、执行权限
  - 文件的时间戳，包括
    - atime，文件上一次打开的时间
    - mtime，文件内容上一次变动的时间
    - ctime，inode上一次变动的时间
  - 链接数，即有多少文件名指向这个inode
    - 1个文件可以有多个名字
  - 文件数据block在磁盘上的位置

inode in Linux

- 硬盘格式化后，每个partition/volume分为两个区域
  - 存放文件数据的数据区
  - 存放文件inode的inode区，即inode table (inode size: 128 bytes, or 256 bytes)
- 1个文件可以有多个文件名，但只有1个inode；每个inode有1个唯一的inode号inode number，作为inode对应的文件的内部唯一标识
  - inode number最大值决定了分区中可创建文件总数
- 文件访问

file name → directory entry(name, inode_num) → inode(..., location on disk,...) → data on disk

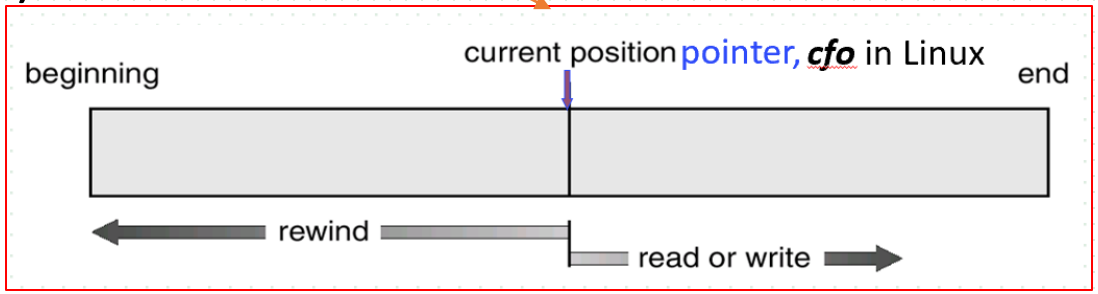
- 硬盘格式化时，在每个partition中，每1K或2K设置1个空inode，便于文件快速创建
  - 文件创建时，直接从free space on disk中，选取1个空inode，分配给新创建文件；新创建文件的存储空间与inode位置相邻或相近

In-memory Structure

- The in-memory information is used for both file-system management performance improvement via **caching**
  - cache some information in on-disk structures into in-memory structures
- Commonly-used in-memory structures are as follows
  - in-memory partition/mount table/安装表
    - information about mounted partition/volume
  - in-memory directory structure cache
    - about recently accessed directory entries
  - system-wide **open-file table**
    - FCBs of opened files
  - per-process **open-file table**
    - pointer to the entry in system-wide open-file table
    - opened files are viewed as the resources used by processes, recorded in PCB
  - buffer
    - hold file data blocks when they are read from disk or written back to disk

Open() vs on-disk and in-memory structures

- On the basis of these data structures, the following file operation are provided
- Creating a new file
  - create the new file's FCB, and insert it into directory in the disk
- When a process opens a file by **Opening**(), OS will
  - search the system-wide open file table to see if the file is already in use by another process
    - 是否已被其它文件打开
  - if it is, OS makes an entry created in this process' ***per-process open-file table*** and this entry points to the system-wide open file, with a pointer to the current location in the file (i.e. cfo in Linux, for *read* or *write* operations)
    - refer to Fig. 11.3(a) for opening a file



Open() vs on-disk and in-memory structures

- if it is not, copy the FCB of the found file on disk into the system-wide open file table in memory, and makes an entry in its ***per-process open-file table***
- open() returns to the process that calls it a pointer to the entry in the per-process open-file table /进程获得指向该进程被每进程打开文件表中对应该文件的表项的指针
  - all file operations are then performed via this pointer, and done on the open-file table
- the name given to the entry is called
  - ***file descriptor*** (文件描述符fd, 非负整数) in Unix/Linux
  - ***file handle*** in Windows
- 多个进程可以打开同一个文件，每个进程在本进程的***per-process open-file table***中创建1个entry，获得1个相对于被打开文件的文件描述符fd。各个进程获得的fd不一样。
- refer to Fig. 11.3(b) for reading a file

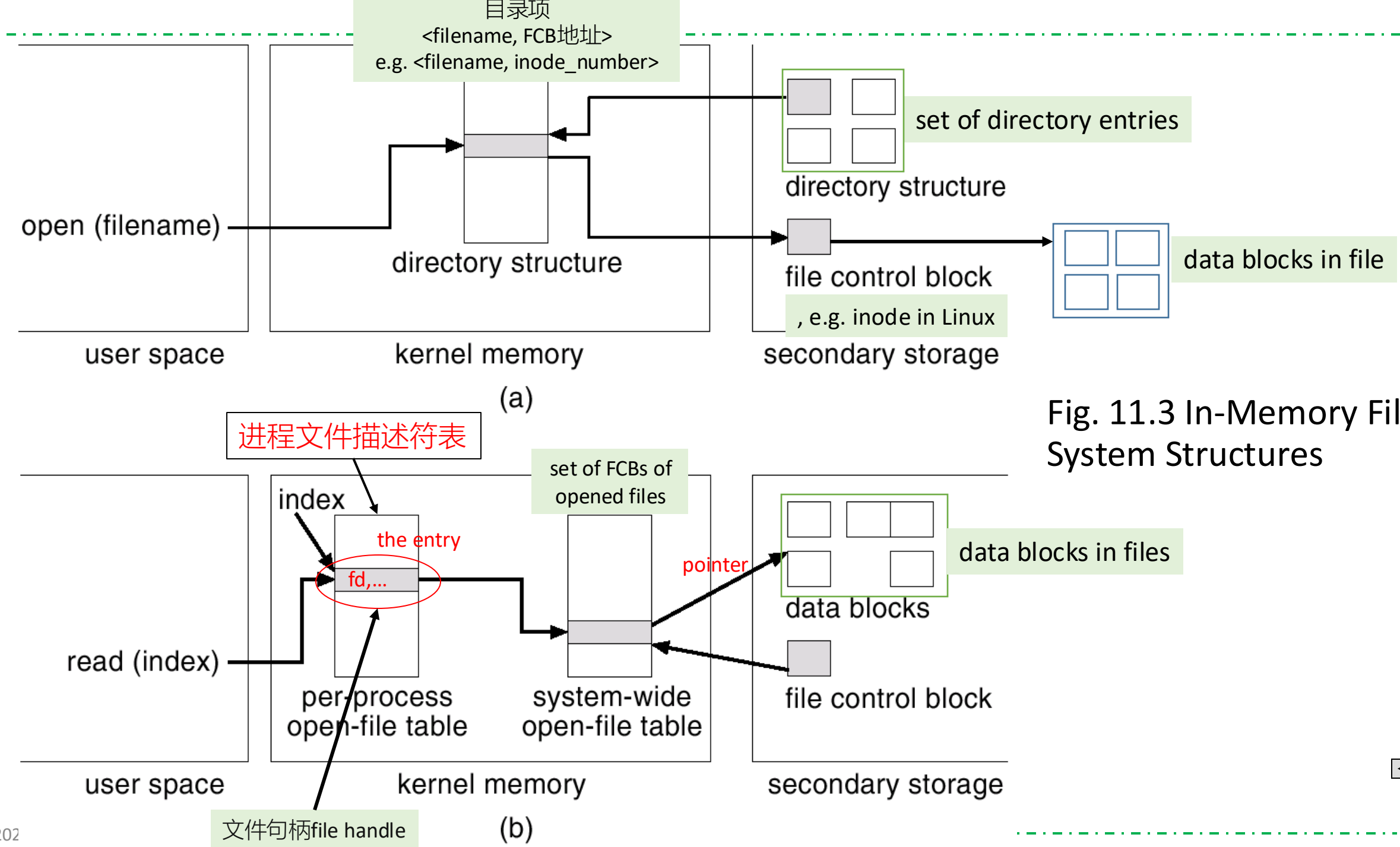


Fig. 11.3 In-Memory File System Structures

硬链接/软链接

- 硬链接(hard link) 
  - Unix/Linux系统允许，允许多个文件名指向同一个inode号码，方便用户用不同的文件名访问同一个文件
  - 1个进程对文件内容进行修改，会影响到通过其它文件名访问文件的进程
  - 删除1个文件名，不影响进程通过另一个文件名访问文件1
- 使用ln命令，创建硬链接：ln 源文件 目标文件
  - 源文件与目标文件的inode号码相同，都指向同一个inode
  - inode节点中的链接数 link count，记录指向该inode的文件名总数，此时增加1
- 删除delete一个文件名，inode节点中的link count减1
  - 当链接数减到0，表明没有文件名指向这个inode，系统回收这个inode号码，以及其对应block区域

硬链接/软链接

- 软链接
  - 文件A和文件B具有各自的inode，其inode号也不一样，但文件A的内容是文件B的路径，文件A指向文件B的文件名
  - 读取文件A时，系统自动将访问者导向文件B，无论打开哪一个文件，最终读取的都是文件B
  - 文件A称为文件B的"软链接"（soft link），或者"符号链接（symbolic link）
- 文件A依赖于文件B而存在，如果删除了文件B，打开文件A就会报错："No such file or directory"
- 软链接与硬链接的不同：文件A指向文件B的文件名，而不是指向文件B的inode号码，文件B的inode link count不会因此发生变化
- ln -s命令创建软链接：
  - ln -s 源文文件或目录 目标文件或目录

Virtual File Systems (VFS)

- Why VFS needed
  - an OS may contain multiple file systems
    - FAT32, UFS, NTFS, ext2, …
  - to support currently multiple types of files, in a generic API
- VFS Functions
  - separate file system generic operations from their implementation by VFS interface, allowing ***transparent*** access to different type of file systems
  - refer to Fig. 11.4 Schematic view of virtual file system
- VFS allows the same system call interface (i.e. POSIX API/system call) to be used for different types of file systems
  - the API is generic and not specific to any particular types of file system

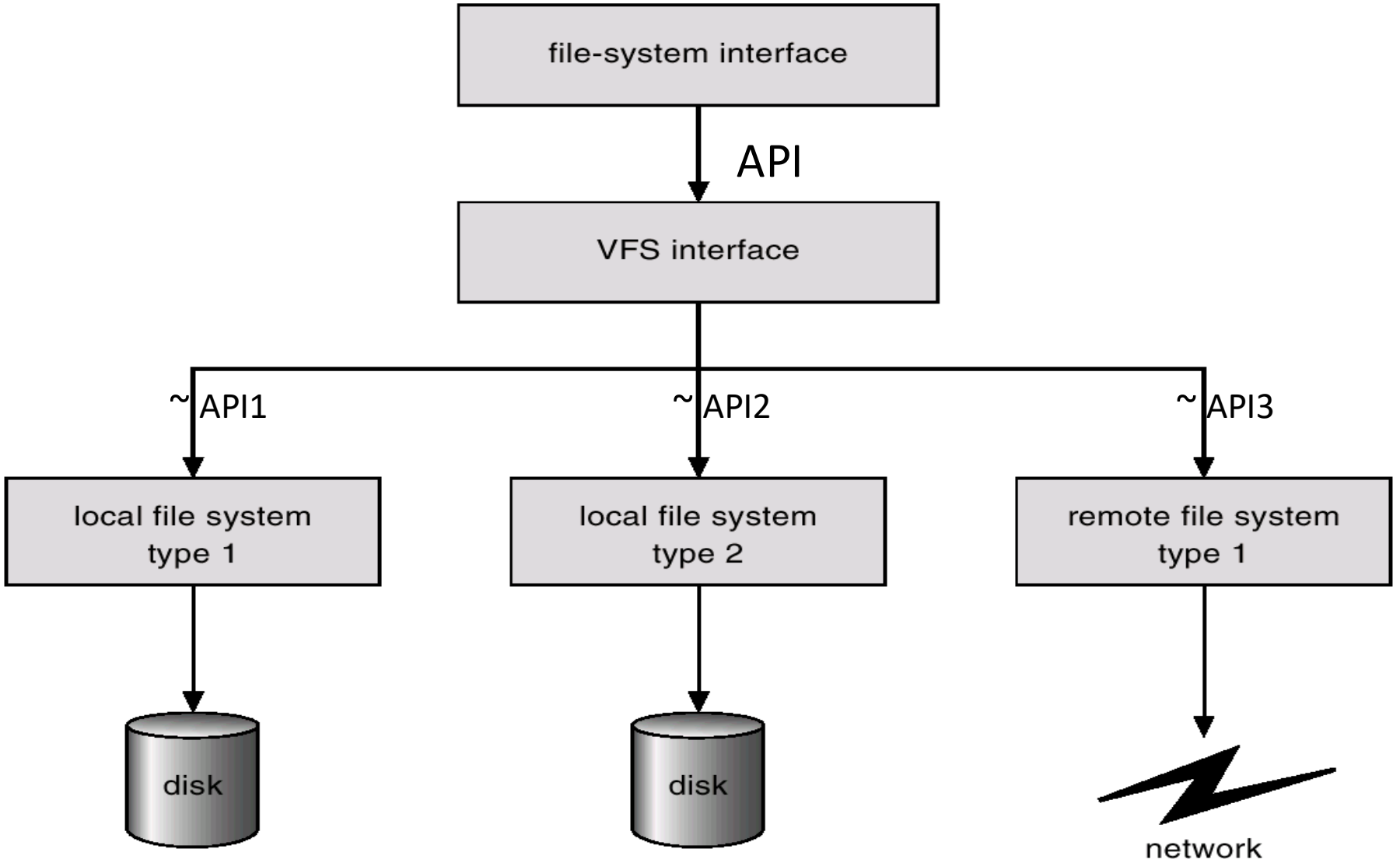
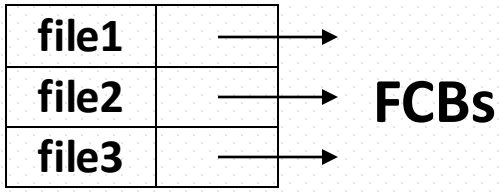


Fig. 11.4 Schematic view of virtual file system

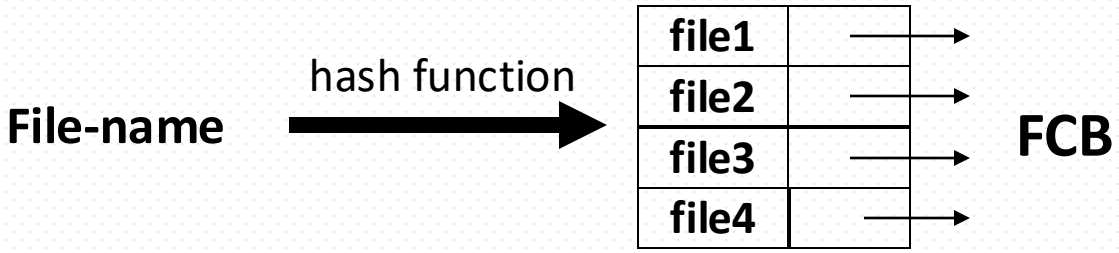
11.3 Directory Implementation

- Directory
  - set of entries
  - entry: file-name → FCB or *pointer* to FCB stored on disk, e.g. Fig.11.3 (a)
- Directory implementation
  - how to organize the entries in the directory for rapid directory access
- **Linear list based implementation**
  - linear list of file names with pointer to the data blocks
  - simple to program, but time-consuming to execute: linear search to find a file
- **Hash-table-based implementation**
  - hash table – linear list with hash data structure
  - decreases directory search time
  - *may be collisions* – situations where two file names hash to the same location

Directory Implementation



(a) list-based implementation



(b) Hash-table-based implementation

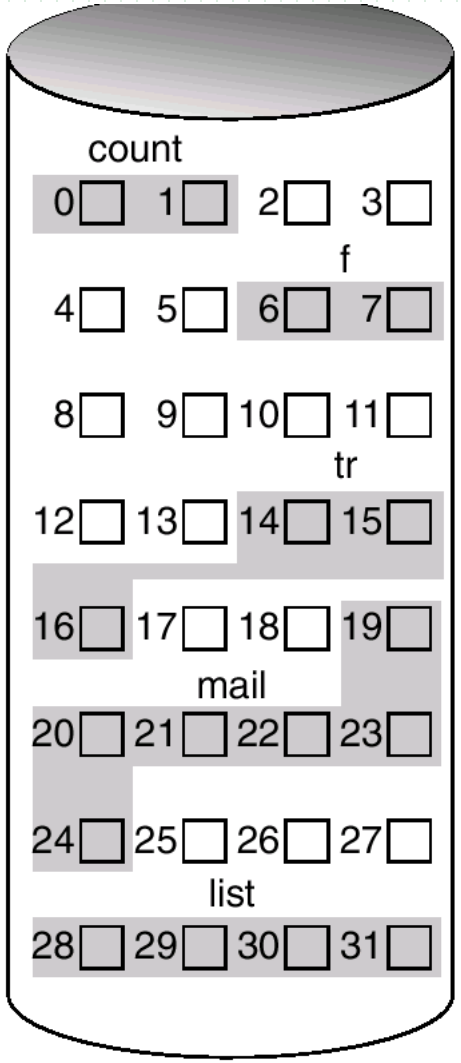
11.4 Allocation Methods

- Allocation methods refer to how disk blocks are allocated for files, that is, the data organization of the file on the disk, and how to access file blocks on disk
  - e.g. block size:2^{10}bytes=1KB, file size 1M=2^{20}B=1024 blocks, how to arrange the blocks on disk

- Three major methods
  - contiguous allocation
  - linked allocation
  - indexed allocation

Contiguous Allocation

- Principle
  - each allocated file occupies a set of contiguous blocks on the disk
  - file address is defined by file's starting location (block #) and length (number of blocks)
  - random/direct access is used
  - refer to Fig. 11.5



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

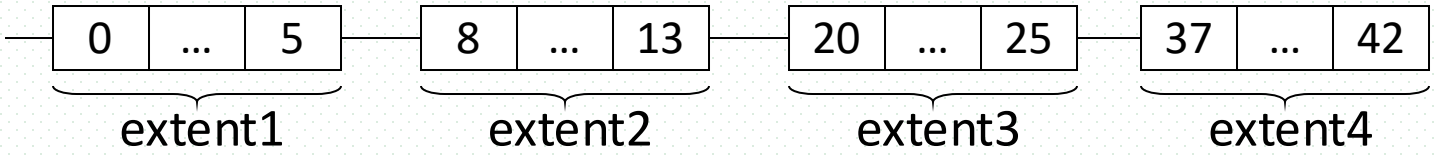
Fig. 11.5 Contiguous Allocation of Disk Space

Contiguous Allocation

- Characteristics
  - wasteful of disk space, external fragmentation
  - when a file is created, the total amount of disk space the file needed must be allocated, so that file size may not grow dynamically
- Contiguous allocation is a particular application of dynamic storage-allocation problem mentioned in §8.3
  - first fit, best fit
- Some older microcomputer systems used contiguous allocation of space on *floppy* disk

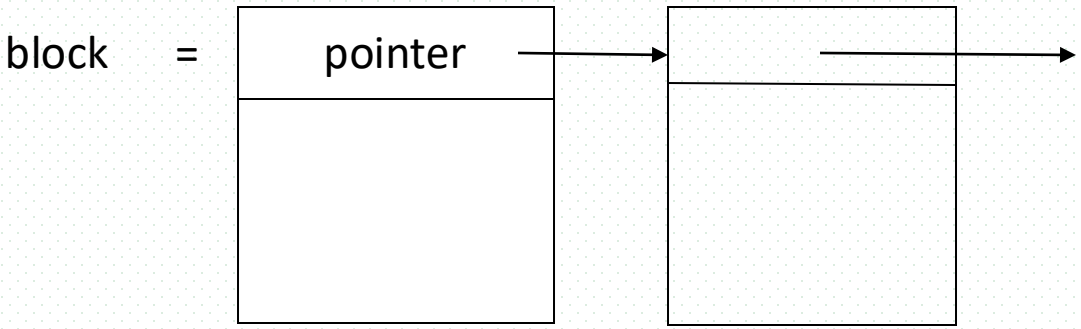
Contiguous Allocation

- An improvement of contiguous allocation, **extent-based** contiguous allocation
  - an **extent** is a contiguous block of disks, with fixed sizes
  - the file is allocated disk space in units of extents, not in the unit of blocks
  - a file consists of one or more extents, and these extents may not be contiguous
  - extent-based contiguous allocation is used in many newer file systems, such as *Veritas File System*
  - e.g. a file with the size of 24 blocks, and 4 extents



Linked Allocation

- Principles
  - each file is organized as a linked list of disk blocks
    - blocks may be scattered anywhere, i.e. noncontiguous on the disk
  - file address: the initial address of the file
  - refer to Fig.11.6



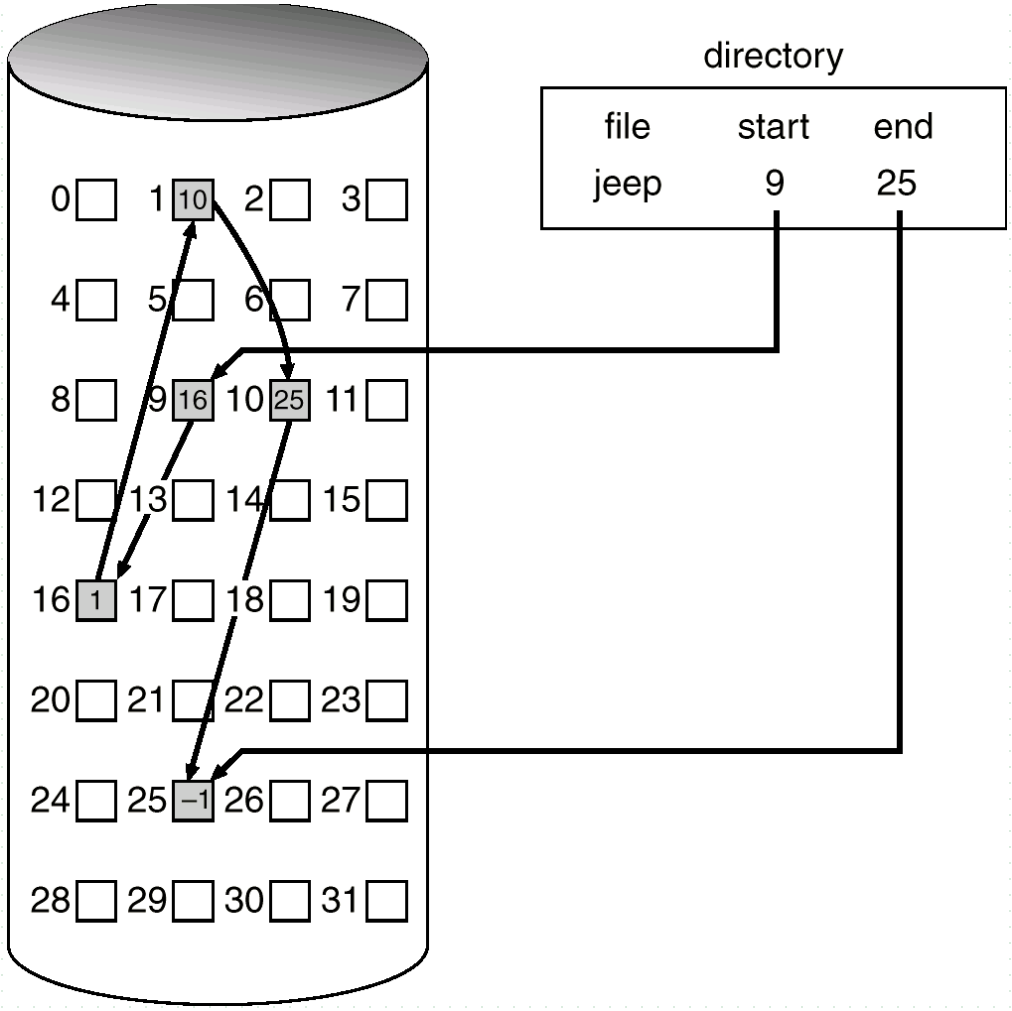


Fig.11.6 Linked Allocation

Linked Allocation

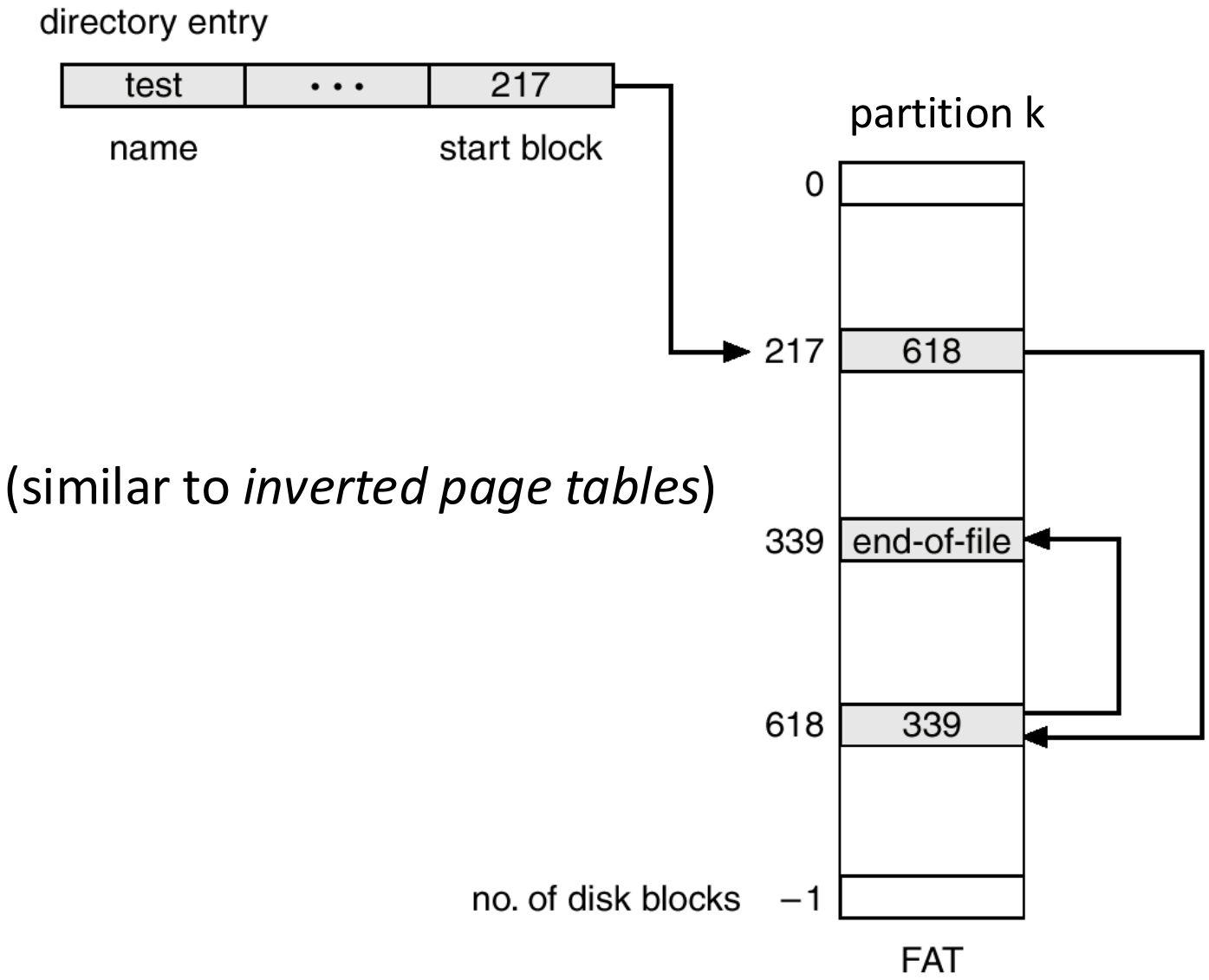
- Characteristics
  - Merits
    - no waste of space, but may have internal fragmentation
  - demerits
    - only sequential access
    - random access not allowed!!
    - space for the pointer may be large

- An implementation— FAT-based linked allocation
  - FAT, File-allocation table
    - data structure for disk-space allocation
    - used by MS-DOS and OS/2

Linked Allocation

- FAT-based allocation
  - refer to Fig. 11.7
  - 1 each partition has one FAT, FAT is located at the beginning section of each partition
  - 2 FAT has one entry for each disk block in the partition, and is indexed by block number
  - 3 FAT is used as a linked list for the files allocated for this partitions
  - 4 the directory entry for a file contain the first block's block# of this file, the FAT entry indexed by that block number contains the next block's block# in the file
  - 5 unused blocks are indicated by a 0 table value

the file ***test*** and its block 217; 618; 339



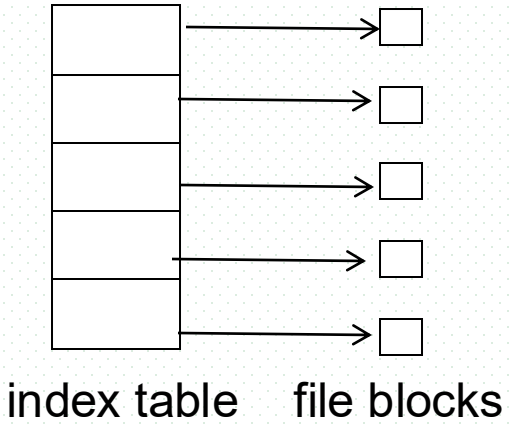
(similar to *inverted page tables*)

例题：
根据指针长度，确定文件最大size，
e.g. 指针长度为4bytes=32 bits，可以指向最多2^{32}个blocks，文件最大size为 2^{32} blocks

Fig. 11.7 File-Allocation Table

Indexed Allocation

- Principles
  - use pointers to indicate the addresses of file blocks, and brings all pointers together into the index block, or index table
  - each file has one or more index blocks, which is a array of disk-block address, the *i*th entry in the index block points to the *i*th block of the file
  - e.g. Fig. 11.8



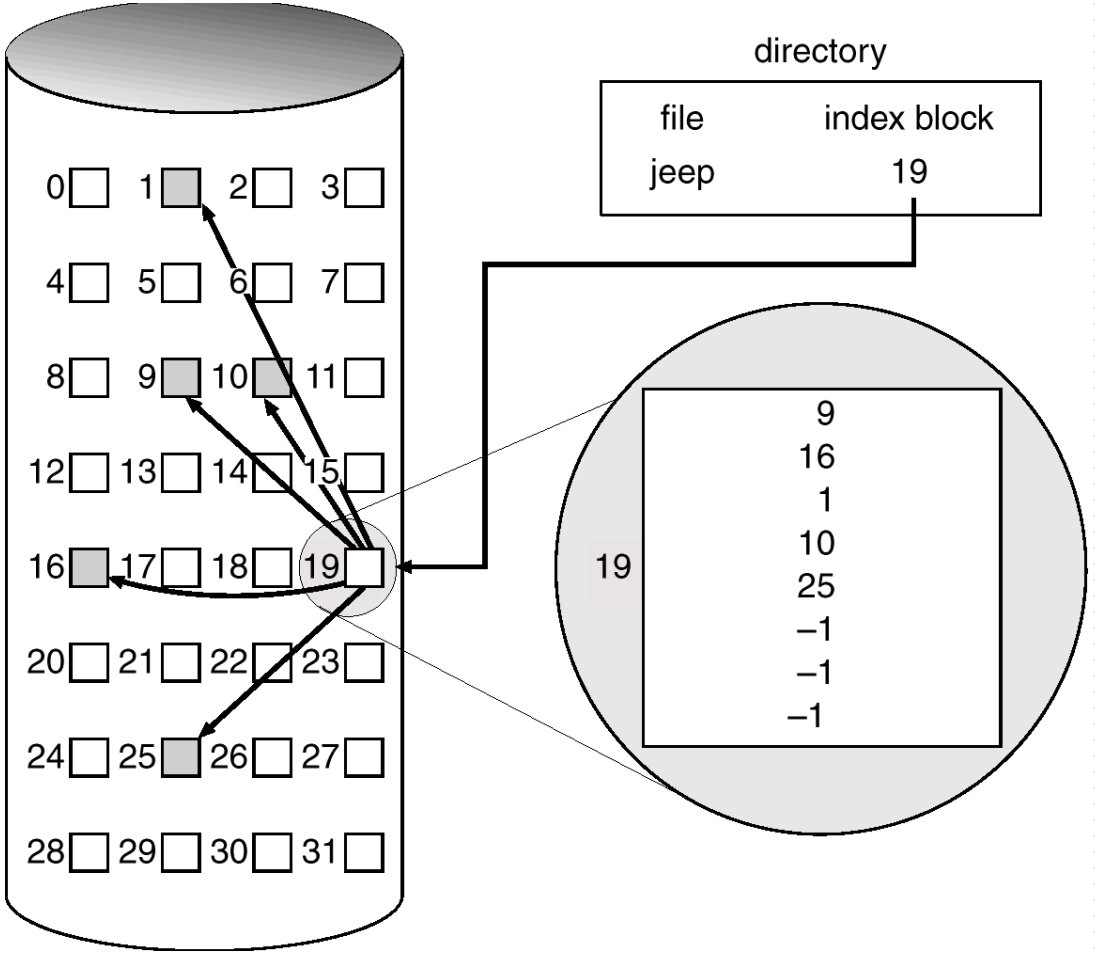


Fig. 11.8 Example of indexed allocation

Indexed Allocation

- Merits
  - random access
  - dynamic access without external fragmentation
- Demerits
  - overhead of index block, e.g.
    - maximum size of file: 256K words
    - block size: 512 words
    - index table: 256K ÷ 512words = 512 words=1 block
- Improvement
  - when the file is large, multi-level indexes can be used
    - similar to multi-level page table
  - combined scheme can be used
  - refer to Fig. 11.9.0 and Fig. 11.9

Indexed Allocation

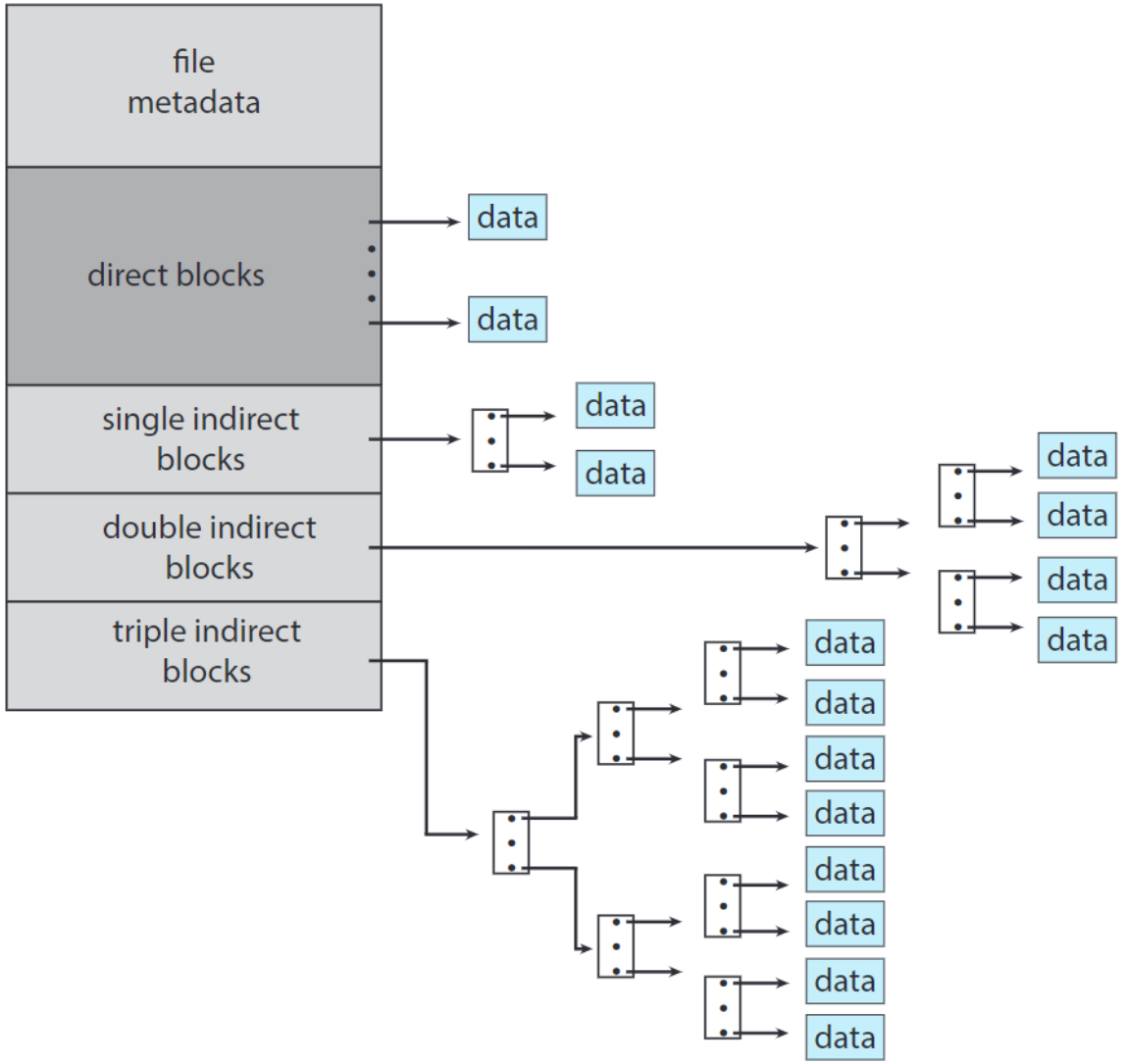


Fig. 11.9.0 Multi-level index

B^+-Tree/B-Tree Index Files【略】

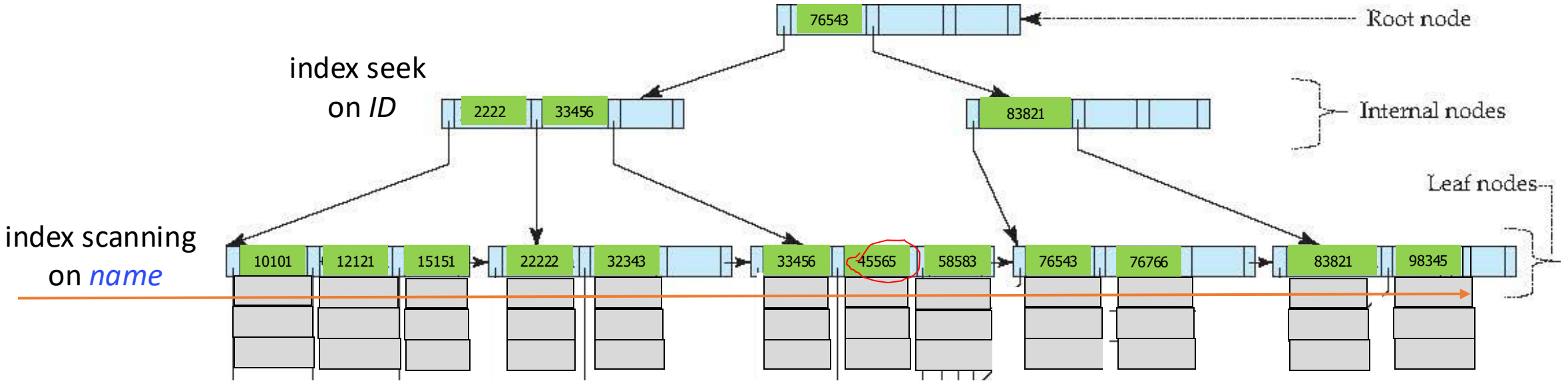
- Wikipedia
  - 1. B tree is a self-balancing tree data structure that keeps data stored and allows searches, sequential accesses, insertions, and deletions in logarithmic time
  - 2. The B tree is a generalization of a binary search tree in that a node can have more than two children.
  - 3. It is commonly used in databases and file systems
- B-tree
  - first proposed in 1970, Bayer R, Mccreight E. Organization and maintenance of large ordered indices[C]// ACM Sigfidet. ACM, 1970:107-141
  - B tree基础上演化产生了B+tree
  - B?: ***B***alanced, ***B***oeing, ***B***ayerB^+-tree indices are an alternative to indexed-sequential file
- Extensively used multi-level index, with advantages
  - automatically reorganizes itself with small, local, changes, in the face of insertions and deletions
  - reorganization of entire file is not required to maintain performance
- Extended in 3D space as R-tree

文件记录直接存储在叶节点

B+-Tree (*n*=4) Index Sequential File

Primary/Clustered index on *ID*

B+树结果不唯一，取决于数据插入顺序




```
create table instructor
      (ID integer, primary key;
       name       varchar(20)
        .....
       )
Or:
 create clustered index IDIndex
        on instructor(ID)
```

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Allocation Methods (cont.)

11.4.4 Performance

- Evaluating allocation methods mentioned above, considering
  - data-block access time
  - storage efficiency, i.e. fragmentations, especially external fragmentations

11.5 Free-Space Management

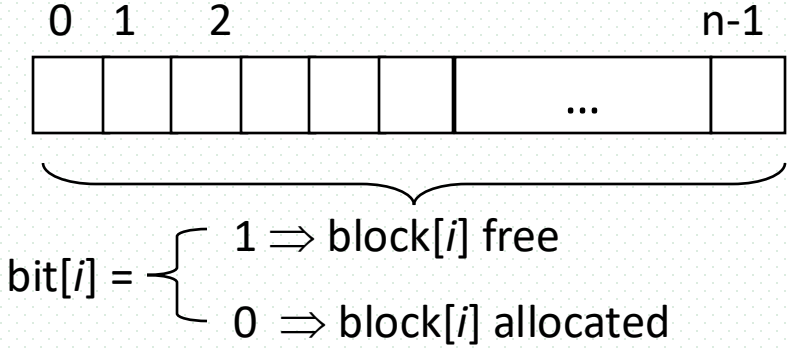
- Free-space management
  - recording free blocks in disk space by ***free-space list***

    e.g. mount table
  - modifying free-space list when files are created or deleted

- To implement *free-space list* based free-space management, four major mechanisms can be used
  - **bit vector/map**
  - linked list
  - grouping
  - counting

Bit Vector/Map

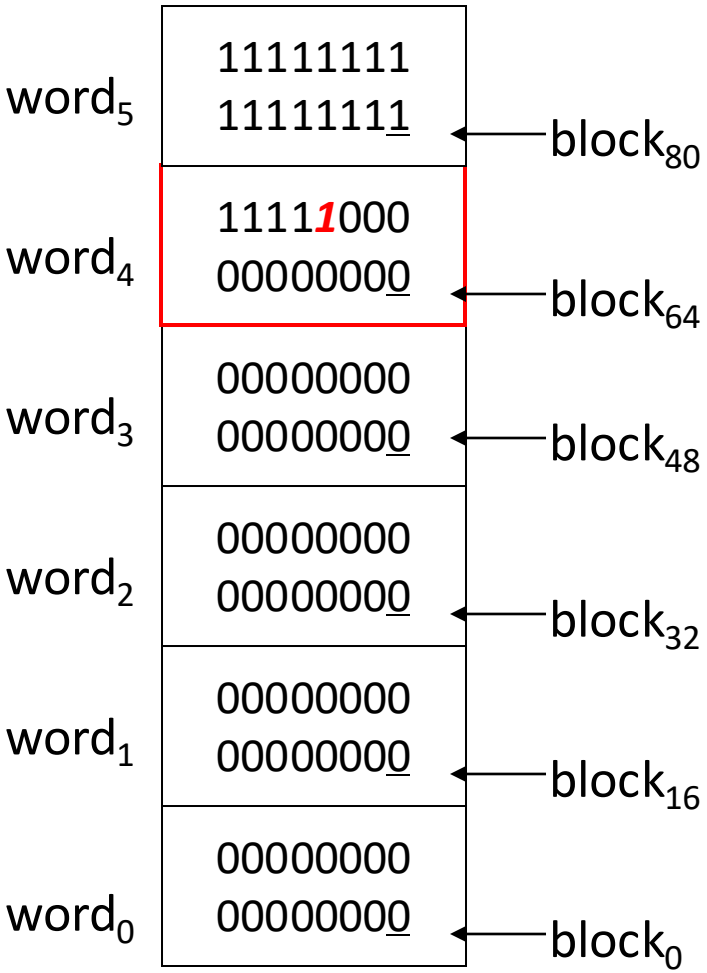
- Principles
  - supposed that there are *n* blocks on disk, the vector/map consists of a set of ***contiguous bytes/words***, with *n*-bits in length
  - each block on the disk is represented by one bit in the vector



  - to find the first free block, OS check sequentially in bit map/vector to find the first non-0 *word* , the block number of founded free block is:

(number of bits per word) *(total number of 0-value words)
+ offset of first 1 bit in the first non-0 *word*

- Fig. 11.5.1 Example of bit vector/map



disk size: total number of blocks
= 6 * 16 = 96

number of bits per word = 16

size of bit map: 6 words

number of 0-value words = 4

first non-0-value word: $word_4$

offset of first 1 bit in $word_4$: 11

first free block is ***block***$_{75}$ ***:***
16 * 4 + 11

bit vector/map: $block_0$ ~ $block_{95}$

Bit Vector/Map

- Used in Macintosh OS, also in DBMS
- Easy to get contiguous files
- Bit map requires extra space
  - e.g.

    block size = 2^{12} bytes=4KB

    disk size = 2^{40} bytes (1024*1 gigabyte)=1T

    $n = 2^{40}/2^{12} = 2^{28}$ bits= 2^{25} bytes

    $= 2^5 * 2^{20}$ bytes (or 32 M bytes)

  so, the size of bit map is 32K, occupies 32M/4KB=8K blocks

Linked list (free list)

- Principles
  - linking together all the free disk blocks
  - keeping a pointer to the first block in a special location on the disk and caching it in memory
  - the first block containing a pointer to the free next block, and so on
  - refer to Fig. 11.10
- Demerits
  - getting contiguous space is not easy
  - traversing the list may be time-consuming

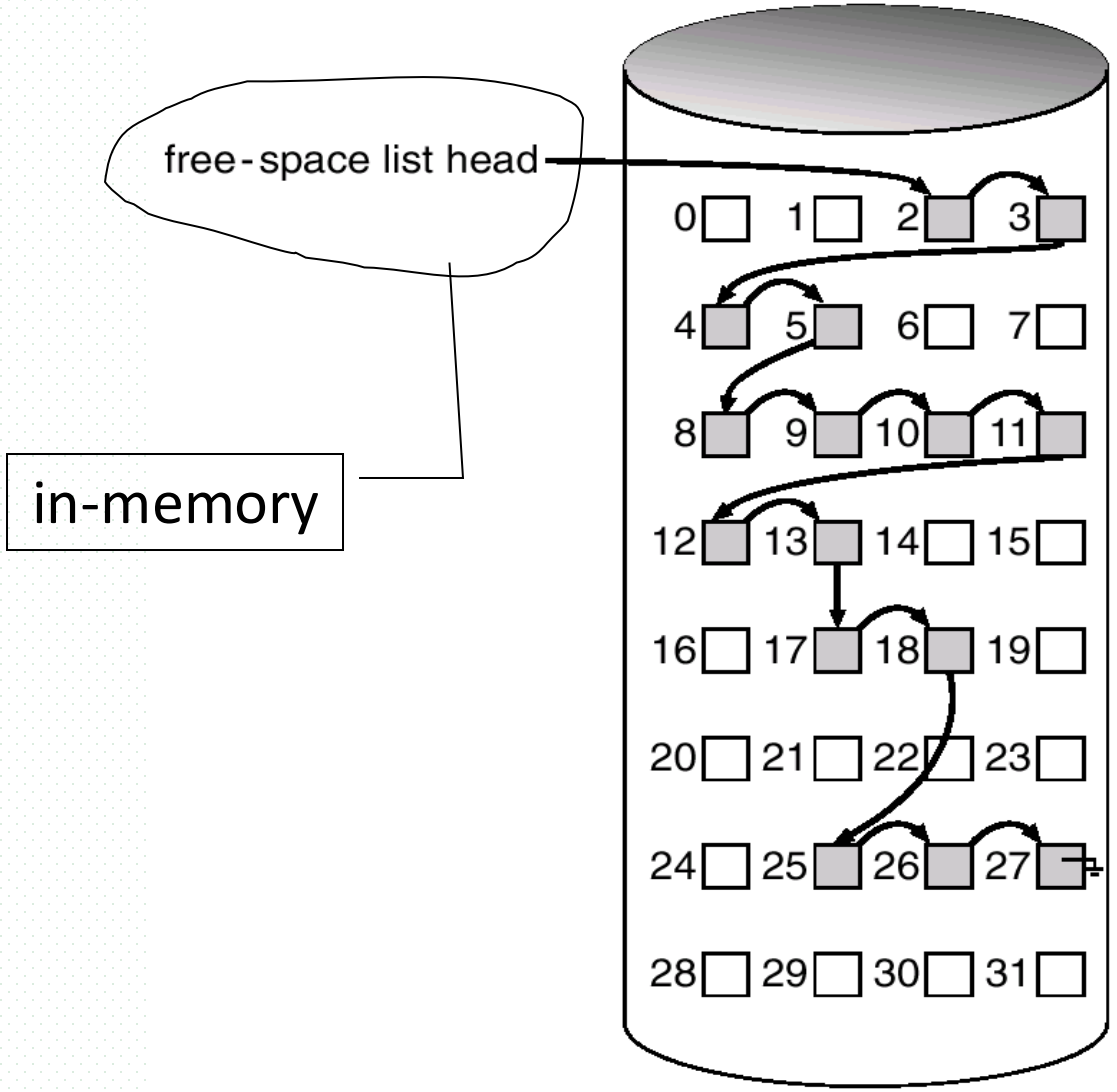
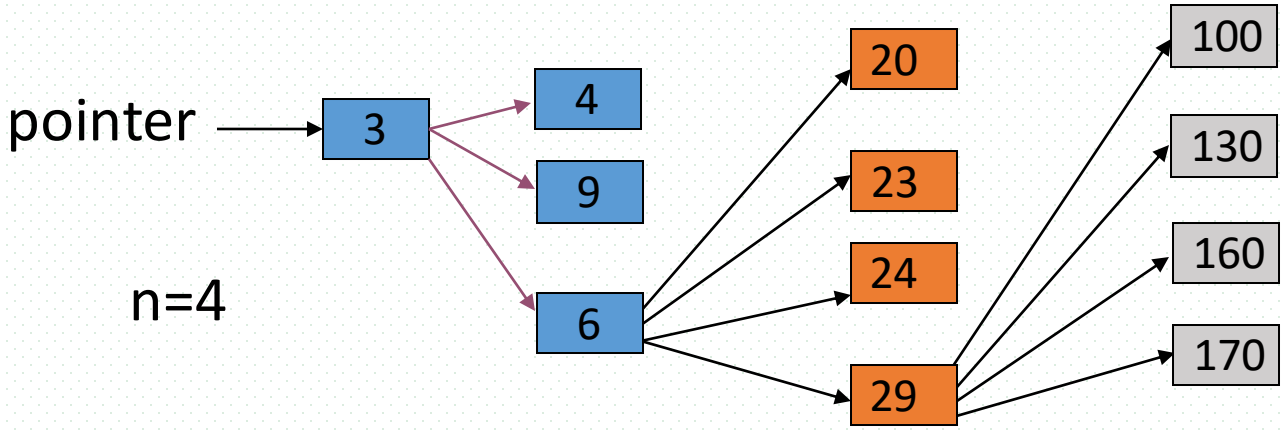


Fig. 11.10 Linked free space list on disk

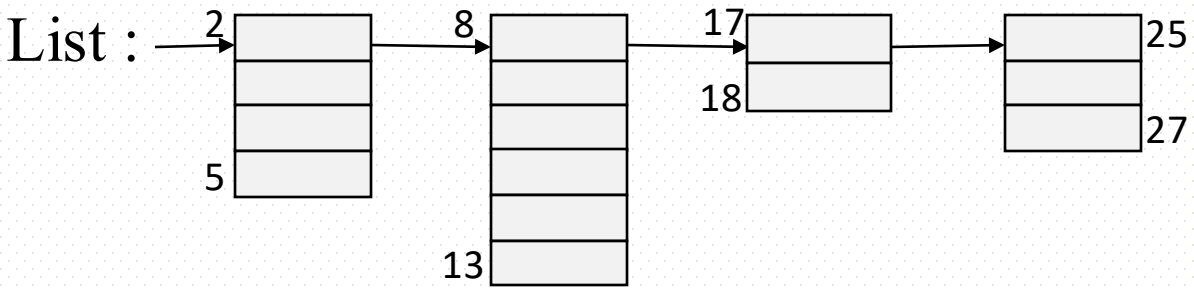
Grouping

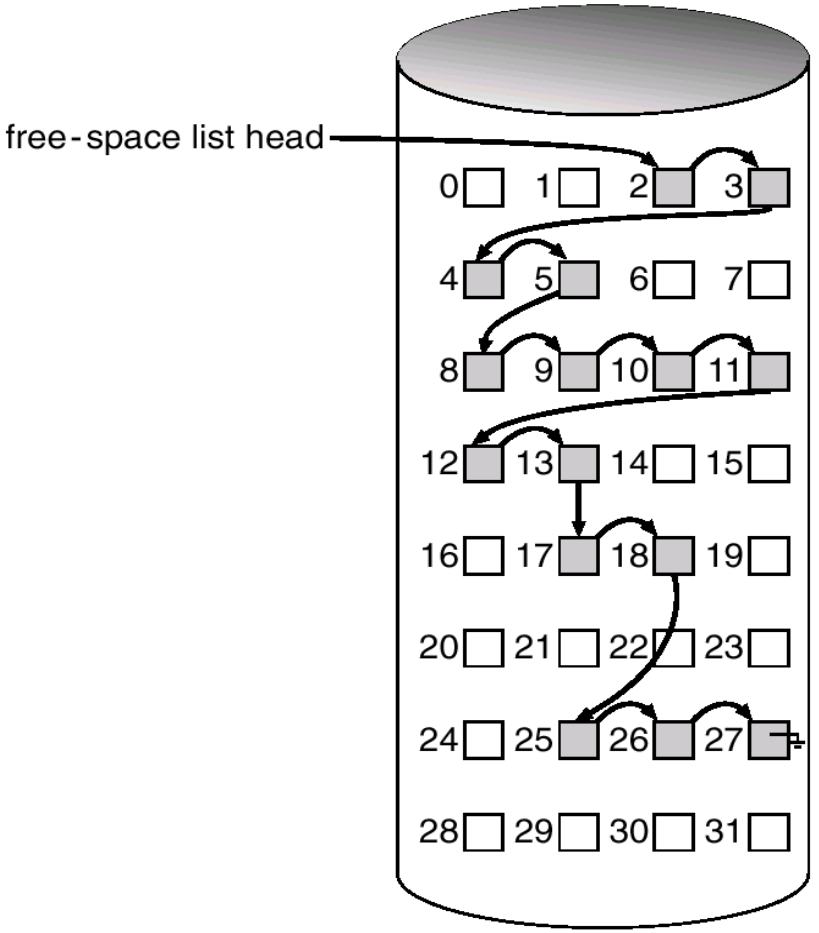
- Principles
  - **n** free blocks are grouped, which may be contiguous, or noncontiguous but adjacent
  - the first block store the address of the other **n-1** free blocks in its group
  - the last free block in one group records the addresses of the blocks in another group



Counting

- Principles
  - several contiguous blocks in one group are allocated or freed simultaneously
  - each entry in free space list records ***the address of*** the first block in the group and ***the number*** of free blocks in this group
  - allocation methods
    - best fit, first fit, worst fit





11.6 Efficiency and Performance

- The disk is a major bottleneck in system performances
- Various disk allocation and directory management mechanisms make effects on efficiency and performance of disk usage
- Techniques used to improve the efficiency and performance

11.6.1 Efficiency

- Efficient usage of disk space ***depends on***
  - disk allocation and directory algorithms
    - e.g.1 contiguous allocation and disk fragments
    - e.g.2 in Unix/Linux, **inode** (索引节点， i.e. FCB) is preallocated on a partition, even for a empty disk; allocation algorithm and free-space algorithm keep a file's data block near that file's ***inode*** block

      waste of space, but efficient

Efficiency

- types of data kept in file's directory entry
  - e.g. "*last access date*" attribute in directory entries

whenever a file is read, this attribute (or the whole directory entry) should be read into memory, be changed and then written back to disk;

inefficient for frequently accessed file

问题：
文件修改/访问时间早于文件创建时间，
why?

Exercises

- 下列优化方法中，可以提高文件访问速度的是

  I. 提前读　　II.为文件分配连续的簇

  III.延迟写　　IV.采用磁盘高速缓存

  A.仅I, II　　B.仅II, III　　C.仅I, III, IV　　D.I, II, III, IV

Exercises

26. 下列选项中，可用于文件系统管理空闲磁盘块的数据结构是_____。

I. 位图　　II. 索引结点　　III. 空闲磁盘块链　　IV. 文件分配表（FAT）

A. 仅 I、II　　B. 仅 I、III、IV　　C. 仅 I、III　　D. 仅 II、III、IV

Exercises

26．某文件系统的簇和磁盘扇区大小分别为1 KB和512 B。若一个文件的大小为1 026 B，则系统分配给该文件的磁盘空间大小是

A．1026 B　　B．1536 B　　C．1538 B　　D．2048 B

Exercises

24. 下列选项中支持文件长度可变，随机访问的磁盘存储空间分配方式是：

A、索引分配；　B、链接分配；　C、连续分配；　D、动态分区分配。

Efficiency and Performance

11.6.2 Performance

- How to improve performances for the selected algorithms
- The approaches include
  - cache
    - object: to reduce the number of access on disks
  - asynchronous write
  - optimizing for sequential access
    - free-behind
    - read-ahead
  - RAM/Virtual disk

Cache

- Method 1— on-board cache in disk controllers
  - track buffer in device controller for storing entire track at a time
  - once a seek is performed, *the track starting at the sector under the disk head* is read into the disk cache
  - the disk controller then transfers any request of the disk sector in this track to OS, and OS may cache the block in the block buffer

on-board cache: set of blocks

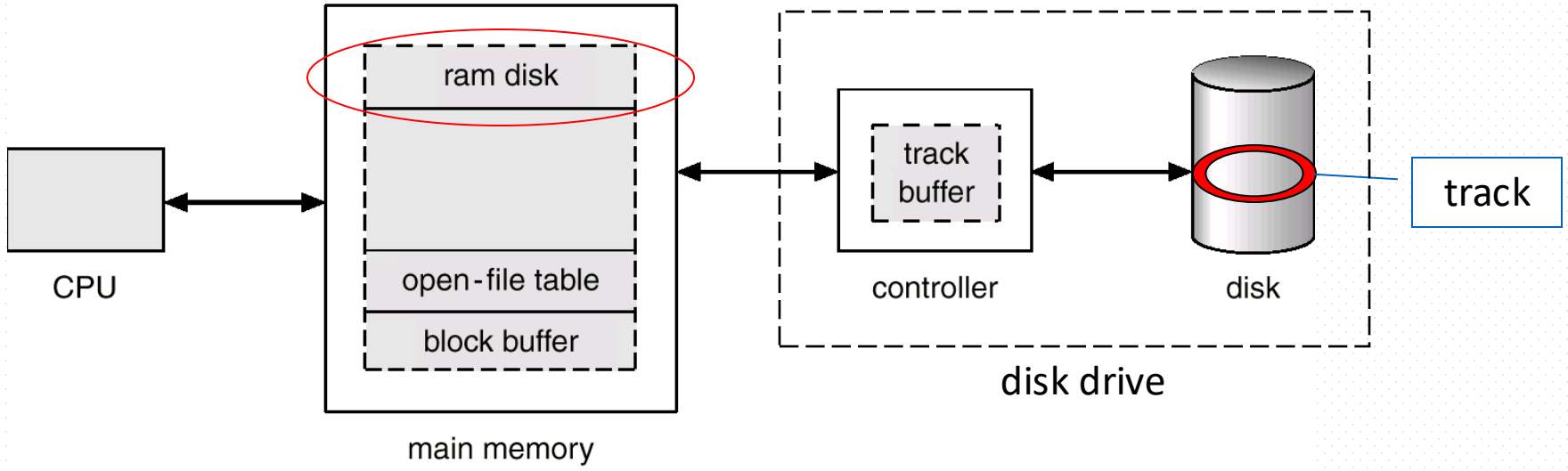


Fig. 11.11.0 on-board in device controller

Fig. Moving-head disk mechanism

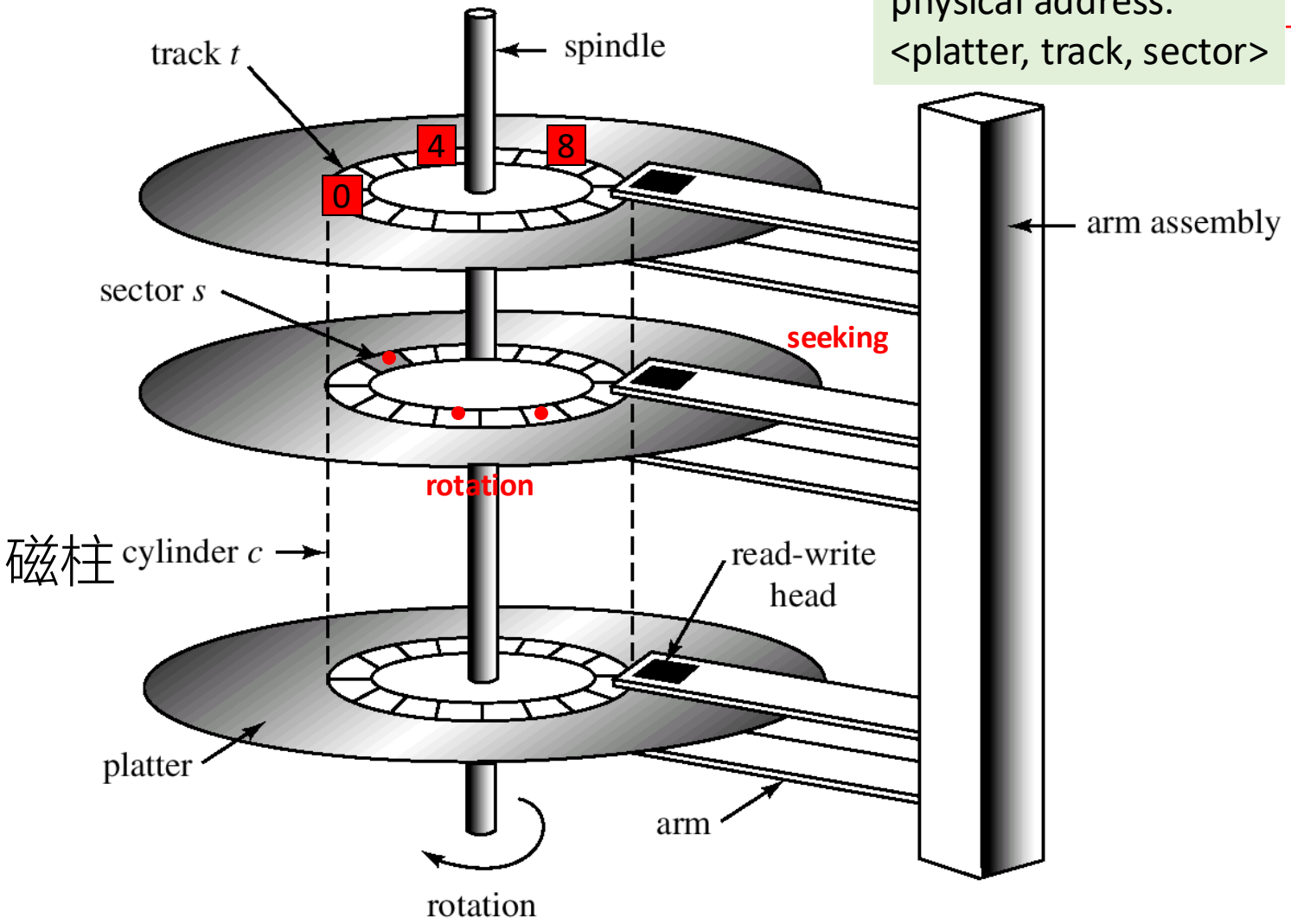
1-dimension block-based organization:

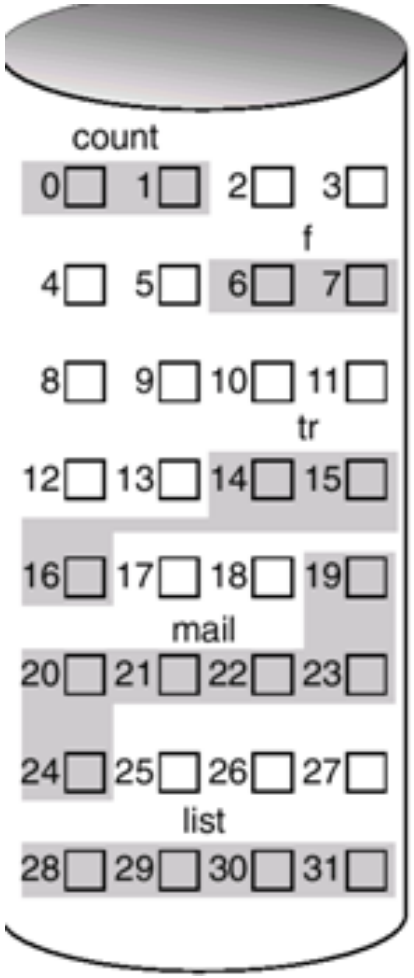
block0, block1, …, block *n*

0	1	2	3	4	5	6	7	8

parallel access to the record 0, 1, 2 by the three r/w heads

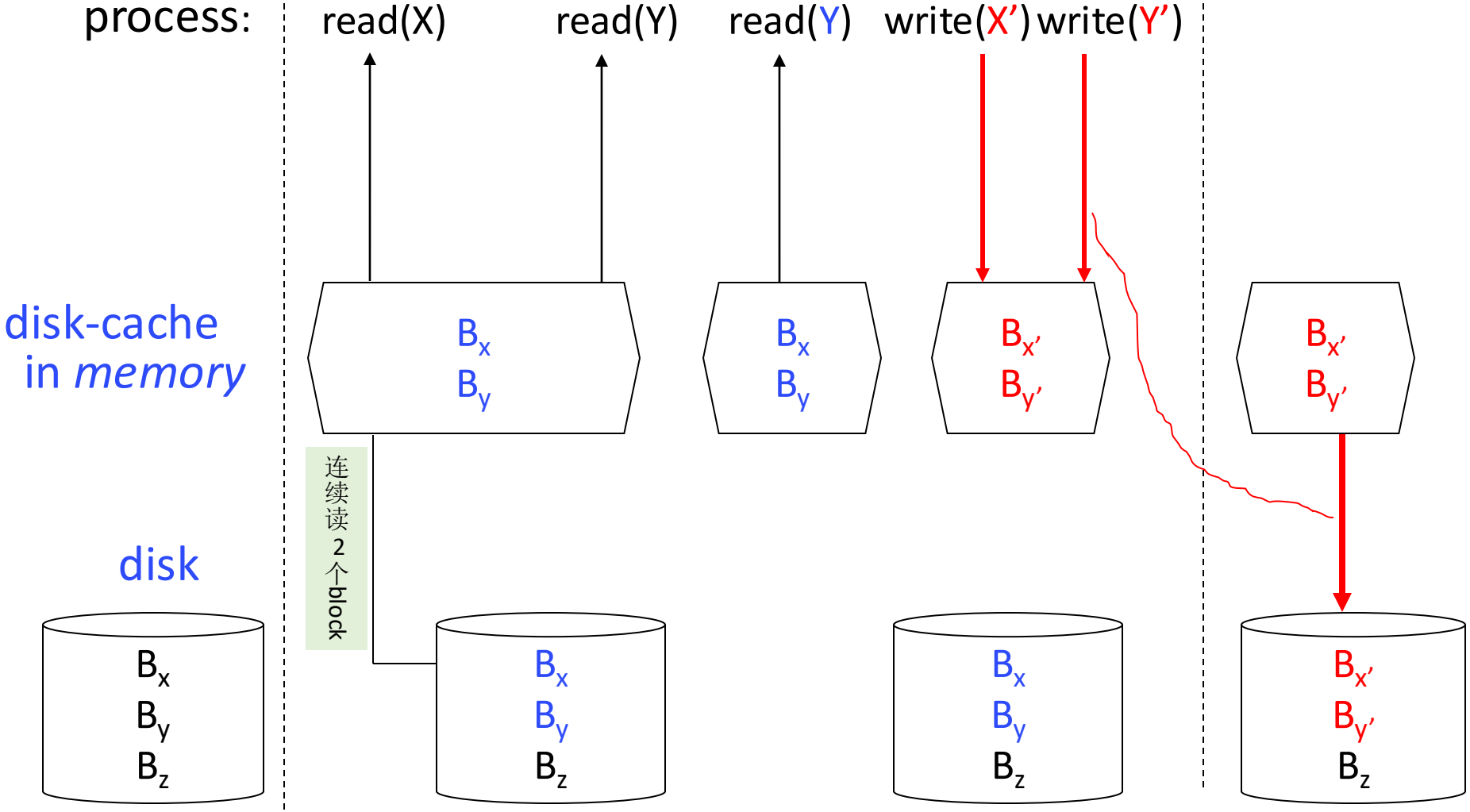
physical address:
<platter, track, sector> → #block





Cache

- **Method 2 — disk cache in main memory**
  - a separate section in main memory, called disk cache or block cache, is set for frequently used blocks, assuming that the blocks in buffer will be used shortly
  - e.g. refer to Fig. 11.11.1
  - a process (e.g. DBS users) writing to disk simply writes into the cache, and OS asynchronously writes the data to disk when convenient
    - this scheme is similar to that in database systems
  - caching is in units of blocks



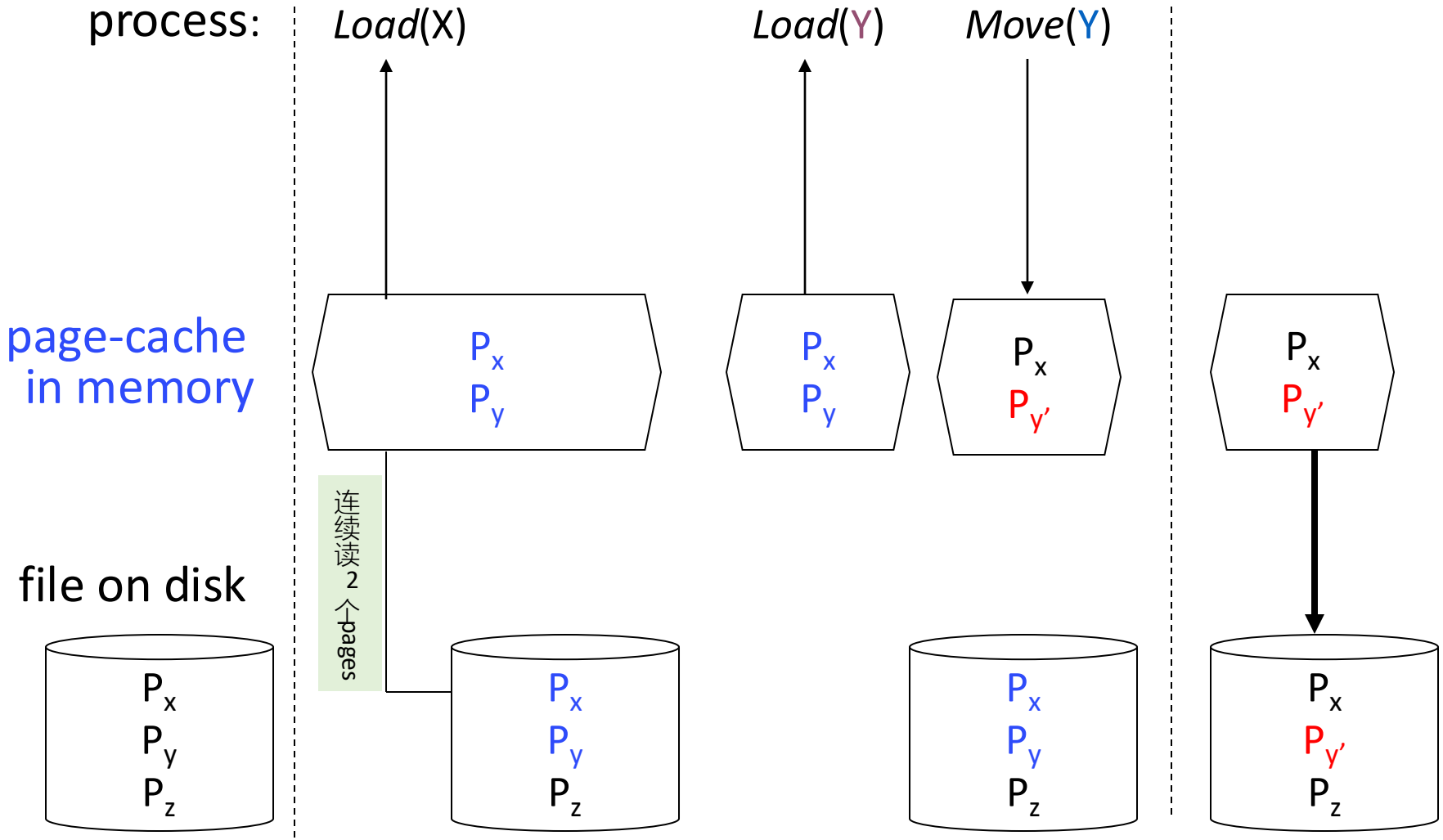
B_x: the block in which data item *X* resides

Fig. 11.11.1 Disk cache and ***asynchronous*** writes

Cache

- **Method 3— page cache**
  - on the basis of virtual memory techniques and memory-mapped file, file data is cached as ***pages*** rather than disk ***blocks***, through page cache
  - page cache, *refer to* and Fig.11.11.2 *in next slide* and Fig.11.11
    - a set of pages/frames in main memory
  - the file is mapped into the process' virtual address space, e.g. *memory-mapped files* in §10.3.2, and the virtual address is used to cache file data

- Unified virtual memory
  - page caching is used to cache both process pages and file pages
  - e.g. Solaris, Linux, Windows NT and 2000



P_x : the page in which data item *X* resides

Fig. 11.11.2 Page cache and asynchronous writes

Cache

- Memory-mapped I/O operations can be implemented by means of page cache
  - memory-mapped I/O
    - the addresses of ***device registers*** in device controllers (i.e., corresponding to disk blocks on disks) are mapped into main memory address space, the access on these memory addresses by ***memory-access instructions*** causes I/O operations on file data through these registers
  - the number of data buffering in memory are reduced

/*磁盘数据直接读入用户空间中的buffer，避免内核空间中的IO数据buffering，减少数据在内存中的缓存次数，提高I/O访问速度

  - whereas the normal I/O operations, i.e., routine I/O is implemented by I/O system calls (e.g. *write( ) and read( )*) on the file system using the *buffer (disk) cache*

淘宝平台双11：
用户态I/O?

Cache

- **Method 4 — Non-unified buffer cache**
  - refer to Fig. 11.11
  - buffer cache/block cache is in units of blocks, and is not visible to processes
  - page cache is in units of pages, and is in process virtual address space, accessed by memory access instructions
  - I/O using write( ) and read( ) system calls go through buffer cache, and is in unit of blocks

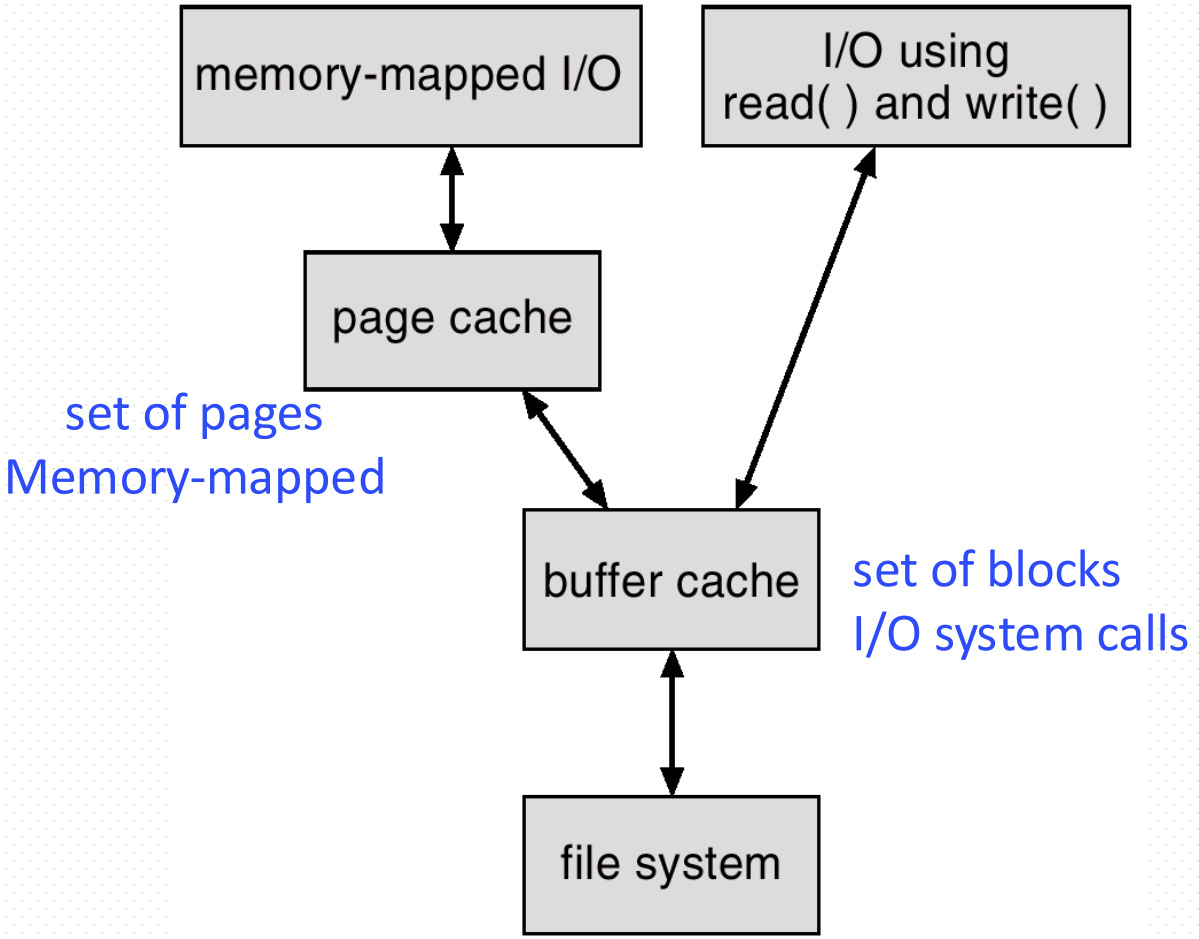


Fig. 11.11 I/O Without a Unified Buffer Cache

Cache

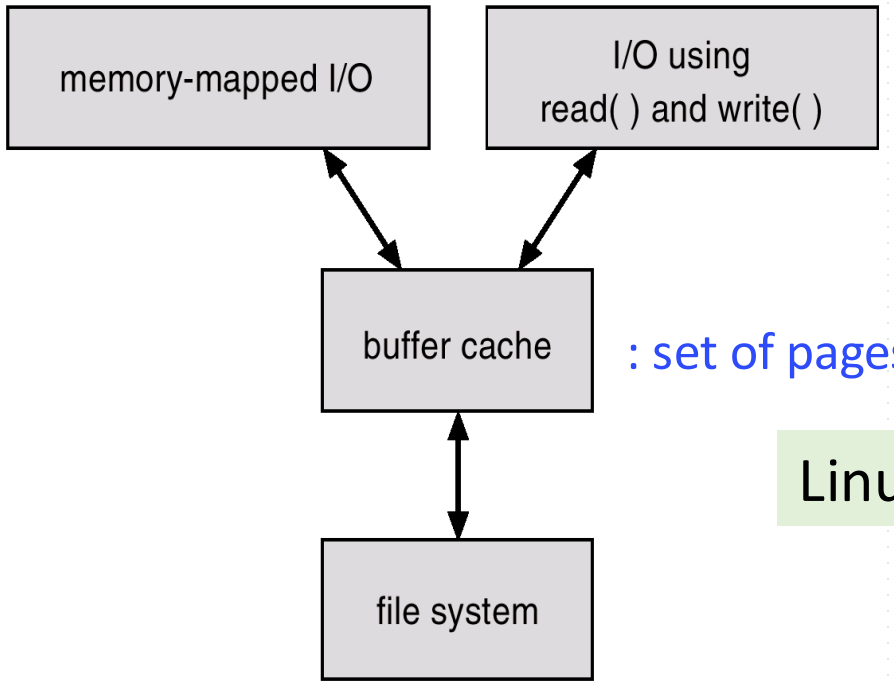
- memory-mapped I/O is on the basis of page cache, in units of pages, proceeding by
  - reading in-disk file blocks into buffer cache
  - because VM can not interface with buffer cache, file blocks in buffer cache are then copied to page cache for processes to access

- demmerits

  double caching, i.e. caching file data twice, can not be avoided, resulting in
  - wasteful of memory and CPU resources
  - inconsistency between two caches

Cache

- **Method 5 — Unified buffer cache**
  - refer to Fig. 11.12
  - page cache is in units of pages, and is visible to both processes and I/O system calls (e.g. write() and read())
  - a unified buffer cache uses the same page cache to cache both memory-mapped pages and ordinary file system I/O

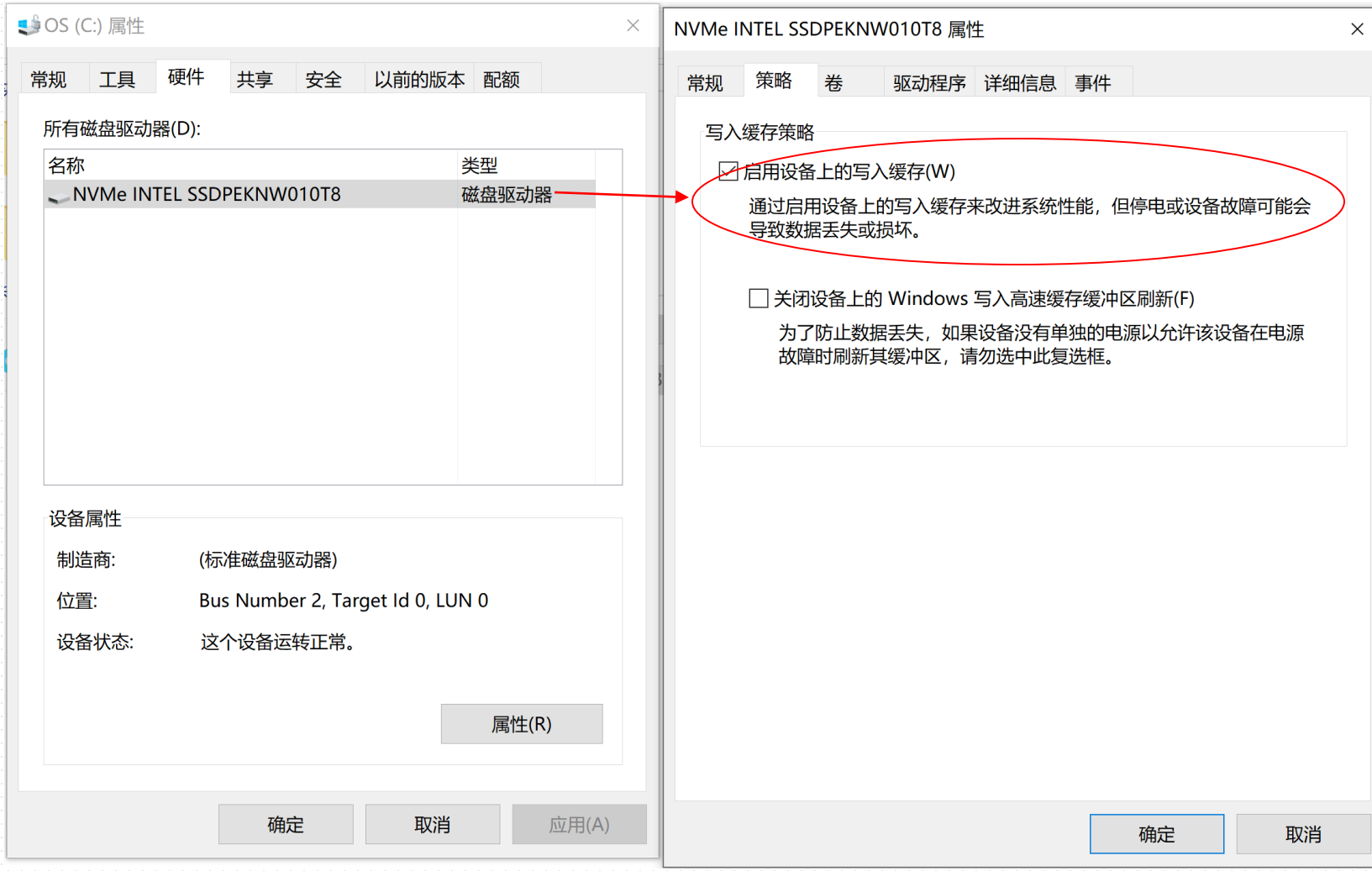


Linux 2.4及其后各版本采用

Fig. 11.12 I/O Using a Unified Buffer/Page Cache

Asynchronous Writes

- A process writing to disks simply writes data into the cache in memory, and the system asynchronously writes the data to disk （刷盘）when convenient
  - e.g. 磁盘写入缓存
  - e.g. DBMS ▷ in Fig.11.1.1



- The process can have very fast data writing

Optimizing for Sequential Access

- Sequential access on file
  - e.g. index scanning on *name* in the table *instructor*(ID, name, deptname, salary) rganized as B+ tree ▷

- Free-behind
  - remove the i-th page from the buffer as soon as the next page (i.e. (i+1)-th page) is requested, because it is supposed that the i-th page may not be used later
- Read-ahead
  - a request page i.e., i-th page, and several subsequent pages , e.g. (i+1)-th page, are read and cached

RAM/Virtual disk (内存盘)

- Improve PC performance by dedicating a section of memory as virtual disk, or RAM disk
  - e.g. RAM disk in Linux
- Fast disk operations are provided for users
- Linux内核自动支持ramdisk
  - 默认创建16 个ramdisks设备，分别是 /dev/ram0 -- /dev/ram15，但未启用，不占用内存空间
  - 使用命令“ls –al /dev/ram*” 可查看ramdisk设备

```
[root]# ls -l /dev/ram*
lrwxrwxrwx    1 root     root            4 Jun 12 00:31 /dev/ram -> ram1
brw-rw----    1 root     disk       1,   0 Jan 30  2003 /dev/ram0
brw-rw----    1 root     disk       1,   1 Jan 30  2003 /dev/ram1
brw-rw----    1 root     disk       1,  10 Jan 30  2003 /dev/ram10
brw-rw----    1 root     disk       1,  11 Jan 30  2003 /dev/ram11
brw-rw----    1 root     disk       1,  12 Jan 30  2003 /dev/ram12
brw-rw----    1 root     disk       1,  13 Jan 30  2003 /dev/ram13
brw-rw----    1 root     disk       1,  14 Jan 30  2003 /dev/ram14
brw-rw----    1 root     disk       1,  15 Jan 30  2003 /dev/ram15
brw-rw----    1 root     disk       1,  16 Jan 30  2003 /dev/ram16
brw-rw----    1 root     disk       1,  17 Jan 30  2003 /dev/ram17
brw-rw----    1 root     disk       1,  18 Jan 30  2003 /dev/ram18
brw-rw----    1 root     disk       1,  19 Jan 30  2003 /dev/ram19
brw-rw----    1 root     disk       1,   2 Jan 30  2003 /dev/ram2
brw-rw----    1 root     disk       1,   3 Jan 30  2003 /dev/ram3
```

查看创建的内存盘

```
1 ls /dev/ram*
```

磁盘设备格式化

```
1 mkfs.ext4 /dev/ram0
```

创建 挂载 设备，进行挂载

```
1 mount /dev/ram0 /log
```

后续卸载

```
1. umount
umount /log
umount /dev/ram0

2. 移出内核
modprobe -r brd
```

Thanks for your attention

北京郵電大學